



PROFESSIONAL ANDROID TERMUX GUIDE

Instant Voice Cloning with OpenVoice and ElevenLabs API

A step-by-step setup guide for recording clean samples, cloning voice tone, generating multilingual speech, integrating playback, and troubleshooting Termux-specific failures.

Target platform	Native Android Termux only, no proot/chroot required
Core tools	Python, Git, FFmpeg, Termux:API, OpenVoice, ElevenLabs SDK
Prepared by	Perplexity Computer
Date	24 April 2026

Workflow stage	Native Termux role
1. Record	Capture clean mic samples with Termux:API.
2. Prepare	Normalize and convert files with FFmpeg.
3. Clone	Run OpenVoice native install attempts or call ElevenLabs IVC.
4. Generate	Create multilingual speech files from text.
5. Play/test	Use mpv/ffplay for clone playback and termux-tts-speak for status prompts.

Consent gate: only clone voices you own or have explicit permission to use. Keep raw samples private and label generated audio in production workflows.

Contents

Contents	2
1. What you are building	3
2. Architecture choices	3
3. Install Termux correctly	4
4. Record clean voice clips	4
5. Prepare audio assets	5
6. Set up OpenVoice	5
6.1 Native Termux install	6
6.2 Native dependency hardening	6
6.3 Checkpoints	7
7. Clone with OpenVoice	7
8. Clone with ElevenLabs IVC	8
9. Generate and play speech	9
10. Test multilingual output	10
11. Fine-tuning decisions	10
12. Troubleshooting	11
13. Build checklist	12

1. What you are building

This guide builds a mobile voice-lab workflow: Termux records reference clips, FFmpeg standardizes them, OpenVoice performs tone-color conversion and style-controlled voice generation, and ElevenLabs Instant Voice Cloning creates an API-hosted clone with the Python SDK. OpenVoice describes its strengths as accurate tone-color cloning, flexible style control for emotion/accent/rhythm/pauses/intonation, and zero-shot cross-lingual cloning; OpenVoice V2 adds better audio quality, native English/Spanish/French/Chinese/Japanese/Korean support, and MIT licensing for commercial use.¹

ElevenLabs IVC is the fast API path from native Termux: its documentation shows creating a clone by calling `elevenlabs.voices.ivc.create` with a name and audio files, then printing the returned `voice_id`.⁴

Keep expectations realistic on Android: ElevenLabs is an API workflow that is well suited to native Termux, while OpenVoice is a PyTorch/MeloTTS stack intended for Linux developers and researchers, so native Termux success depends on whether your device and current packages can satisfy the Python and native-library dependencies.²

Ethics gate: clone only your own voice or a voice you have explicit rights to use. Keep consent records, do not publish deceptive generated speech, and label AI-generated voice assets in production workflows.

1. OpenVoice GitHub README: <https://github.com/myshell-ai/OpenVoice>

2. OpenVoice usage guide: <https://github.com/myshell-ai/OpenVoice/blob/main/docs/USAGE.md>

4. ElevenLabs IVC quickstart: <https://elevenlabs.io/docs/eleven-api/guides/how-to/voices/instant-voice-cloning>

5. ElevenLabs voice cloning overview: <https://elevenlabs.io/docs/eleven-creative/voices/voice-cloning>

2. Architecture choices

Path	Best use	Runs where	Strengths	Limits
OpenVoice V2	Granular style control, experiments, local/private pipelines	Native Termux Python venv when dependencies build	Tone-color cloning, emotion/accent/rhythm/intonation controls, multilingual generation	PyTorch/NumPy/librosa/MeloTTS dependencies can be heavy on Android
ElevenLabs IVC	Fast clone creation and API production	Native Termux Python SDK	Short-sample cloning, hosted voice ID, simple TTS generation	Cloud dependency, account/API limits, less low-level style control than OpenVoice
ElevenLabs PVC	High-fidelity production voice model	Python SDK/API workflow from Termux	Dedicated fine-tuned model for unique voices/accents	Creator+ feature, larger clean dataset, verification and training time

ElevenLabs distinguishes IVC and PVC as different methods: IVC creates clones from shorter samples near-instantly without training a custom model, while PVC fine-tunes a dedicated model and is better for unique voices or production quality requirements.⁵

The ElevenLabs overview states that PVC is available on Creator+ plans and typically requires a larger set of voice data, with fine-tuning often taking several hours depending on queue and other factors.⁵

2. OpenVoice usage guide: <https://github.com/myshell-ai/OpenVoice/blob/main/docs/USAGE.md>

5. ElevenLabs voice cloning overview: <https://elevenlabs.io/docs/eleven-creative/voices/voice-cloning>

3. Install Termux correctly

Install Termux and Termux:API from the same source, preferably F-Droid or the matching GitHub release channel. The Termux project warns that the app and plugins use the same Android shared user ID and must be signed by the same source, so mixing F-Droid Termux with a GitHub or Play Store Termux:API plugin can break installation or API permissions.⁸

Termux package management uses apt/dpkg under the hood, but the wiki recommends using pkg because it wraps apt, updates when needed, and handles mirror selection; it also recommends upgrading packages before installing new packages.⁹

```
# In fresh Termux
pkg update
pkg upgrade

# Core voice-lab packages
pkg install python git ffmpeg termux-api clang make cmake pkg-config libsndfile

# Storage access for sample files and exports
termux-setup-storage

# Confirm toolchain
python --version
git --version
ffmpeg -version | head -n 1
```

Do not add Debian or Ubuntu package repositories to native Termux. Termux explicitly does not support Debian/Ubuntu packages in its sources list or manual .deb installs, because its Android runtime and filesystem layout differ from conventional Linux distributions.

8. Termux app installation notes: <https://github.com/termux/termux-app>

9. Termux package management: https://wiki.termux.com/wiki/Package_Management

4. Record clean voice clips

Termux microphone access requires the Termux:API app plus the termux-api package. The microphone command supports recording to a file with -f, a duration limit with -l, encoder selection with -e, bitrate with -b, sample rate with -r, channel count with -c, status with -i, and quitting with -q.¹⁰

Native Termux capture sequence	Purpose
pkg install python git ffmpeg termux-api	Install the minimum toolchain.
termux-microphone-record -f samples/ref_01.m4a -l 25 -e aac -r 44100 -c 1	Record a short mono reference clip.
ffmpeg -i samples/ref_01.m4a -ar 22050 -ac 1 samples/ref_01_22k.wav	Prepare an OpenVoice-style WAV.

```
mkdir -p ~/voice-lab/samples ~/voice-lab/out
cd ~/voice-lab
```

```
# Record 20-30 seconds of dry, single-speaker audio.
# Speak naturally: include normal cadence, pauses, and a few expressive lines.
termux-microphone-record -f samples/ref_01.m4a -l 25 -e aac -r 44100 -c 1
```

```
# Optional: inspect active/last recording state.
termux-microphone-record -i
```

```
# Convert for OpenVoice-style pipelines: mono, 22.05 kHz WAV.
ffmpeg -y -i samples/ref_01.m4a -ar 22050 -ac 1 samples/ref_01_22k.wav
```

```
# Convert for ElevenLabs upload or archive: mono, 44.1 kHz WAV.
ffmpeg -y -i samples/ref_01.m4a -ar 44100 -ac 1 samples/ref_01_44k.wav
```

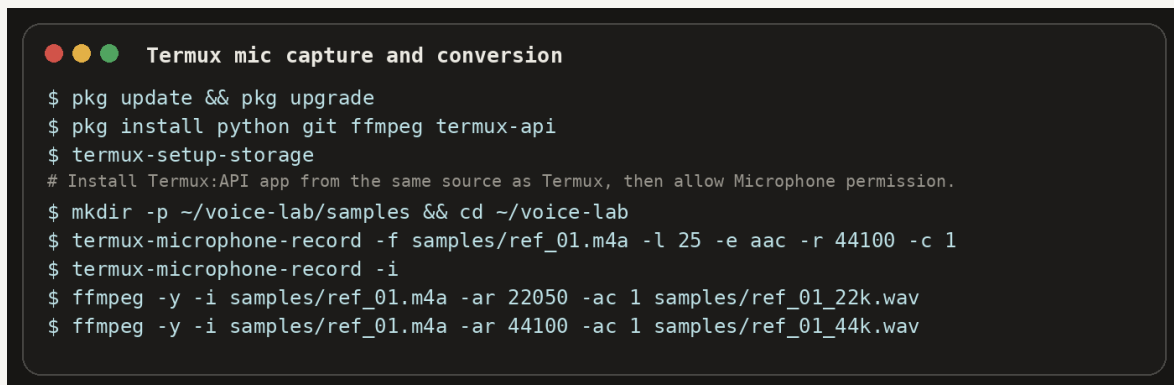


Figure 3. Terminal-style screenshot of the recommended Termux microphone capture flow.

Clip recipe: record three to five clips of 20–45 seconds each. Use a quiet room, hold the phone steady, avoid music/background speech, and include the exact style you want the clone to learn: calm narration, urgent alert voice, trading commentary, or multilingual phrases.

10. Termux microphone record: <https://wiki.termux.com/wiki/Termux-microphone-record>

5. Prepare audio assets

Voice cloning is only as good as the reference audio. Treat the phone microphone as a field recorder: keep the mouth 15–25 cm from the device, avoid clipping, record in a wardrobe/soft room if possible, and use consistent mic position across clips.

```
# Check sample metadata
ffprobe -hide_banner samples/ref_01_22k.wav

# Remove long leading/trailing silence while preserving natural pauses
ffmpeg -y -i samples/ref_01_22k.wav -af "silenceremove=start_periods=1:
start_threshold=-45dB:detection=peak" samples/ref_01_trim.wav

# Normalize safely for analysis/upload, not aggressively
ffmpeg -y -i samples/ref_01_trim.wav -af "loudnorm=I=-18:TP=-3:LRA=11" samples/
ref_01_norm.wav

# Quick listen in Termux
ffplay -nodisp -autoexit samples/ref_01_norm.wav
```

Target	Suggested format	Why
OpenVoice reference	WAV, mono, 22.05 kHz	Many OpenVoice examples and converter pipelines operate around 22.05 kHz audio handling.
ElevenLabs upload	MP3, M4A, or WAV from clean source	The IVC SDK example uploads audio files as binary streams and says more files can improve the clone.
Archive master	WAV, mono, 44.1 or 48 kHz	Keeps a higher-quality source for future PVC/fine-tuning workflows.

4. ElevenLabs IVC quickstart: <https://elevenlabs.io/docs/eleven-api/guides/how-to/voices/instant-voice-cloning>

5. ElevenLabs voice cloning overview: <https://elevenlabs.io/docs/eleven-creative/voices/voice-cloning>

6. Set up OpenVoice

The official OpenVoice usage guide targets Linux/Python/PyTorch users and starts with Python 3.9 in a conda environment, cloning `mysheLL-ai/OpenVoice`, and installing the repo in editable mode with `pip install -e ..`²

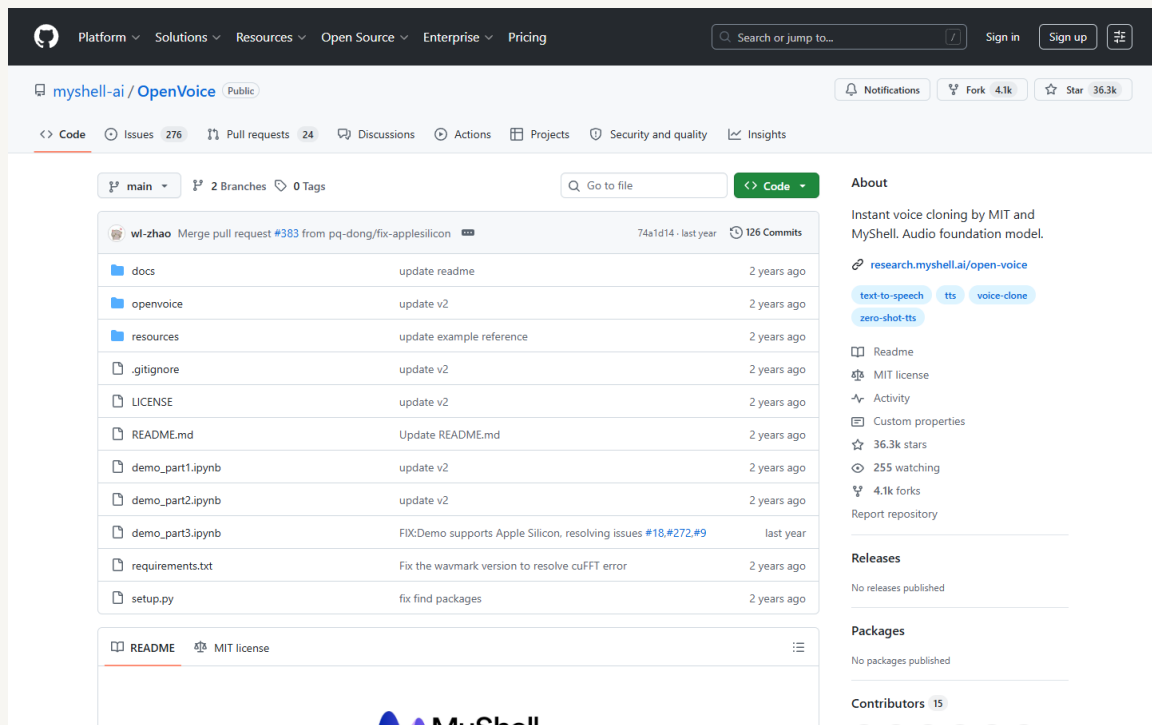


Figure 4. Official OpenVoice GitHub repository.

6.1 Native Termux install

Use this path directly on the phone. Expect that PyTorch, NumPy, librosa, and native audio dependencies may require retries on some Android devices; the troubleshooting section keeps the fixes inside native Termux.

```
cd ~/voice-lab
python -m venv .venv-openvoice
source .venv-openvoice/bin/activate
pip install --upgrade pip wheel setuptools

git clone https://github.com/myshell-ai/OpenVoice.git
cd OpenVoice
pip install -e .

# OpenVoice V2 also needs MeloTTS and Unidic, per official usage docs.
pip install git+https://github.com/myshell-ai/MeloTTS.git
python -m unidic download
```

6.2 Native dependency hardening

If the first install fails, stay in native Termux and harden the build environment. Upgrade the package index, install compilers and media libraries, rebuild the virtual environment, and prefer prebuilt wheels where available.

```
deactivate 2>/dev/null || true
cd ~/voice-lab

# Refresh native Termux packages and build tooling.
pkg update && pkg upgrade
pkg install clang make cmake pkg-config rust binutils-is-llvm
pkg install libsndfile sox ffmpeg

# Recreate a clean native venv.
rm -rf .venv-openvoice
python -m venv .venv-openvoice
source .venv-openvoice/bin/activate
pip install --upgrade pip wheel setuptools

cd ~/voice-lab/OpenVoice
pip install -e .
pip install git+https://github.com/myshell-ai/MeloTTS.git
python -m unidic download
```

6.3 Checkpoints

For V1, the official usage guide says to download the checkpoint and extract it to the checkpoints folder; for V2, download the V2 checkpoint and extract it to checkpoints_v2.²

```
# From inside OpenVoice repo after downloading the official checkpoint zip:
mkdir -p checkpoints_v2
unzip /path/to/checkpoints_v2*.zip -d checkpoints_v2
```

```
# Expected high-level layout for V2 examples:
find checkpoints_v2 -maxdepth 3 -type f | head
```

Native Termux reality check: OpenVoice can be slow on CPU-only phones and may fail where Android-native wheels are unavailable. Keep the setup reproducible, log every package error, and use ElevenLabs IVC as the reliable native-Termux baseline while you work through OpenVoice dependency issues.

2. OpenVoice usage guide: <https://github.com/myshell-ai/OpenVoice/blob/main/docs/USAGE.md>

7. Clone with OpenVoice

OpenVoice works conceptually as a base TTS voice plus a tone-color converter: the base model creates speech in a chosen style, then the converter applies the reference speaker's tone color. The official docs point to demo_part1.ipynb for flexible style control, demo_part2.ipynb for cross-lingual cloning, and demo_part3.ipynb for V2 usage.²

```
# Minimal adaptation pattern for demo_part3.ipynb.
# Run from the OpenVoice repository after checkpoints are in place.

from melo.api import TTS
from openvoice import se_extractor
from openvoice.api import ToneColorConverter
import torch, os

ckpt_converter = "checkpoints_v2/converter"
device = "cuda:0" if torch.cuda.is_available() else "cpu"
output_dir = "../out"
os.makedirs(output_dir, exist_ok=True)

tone_color_converter = ToneColorConverter(
    f"{ckpt_converter}/config.json",
    device=device,
)
tone_color_converter.load_ckpt(f"{ckpt_converter}/checkpoint.pth")

reference_speaker = "../samples/ref_01_norm.wav"
target_se, audio_name = se_extractor.get_se(
    reference_speaker,
    tone_color_converter,
    vad=True,
)

texts = {
    "EN_NEWEST": "This is a calm trading desk update from my Android voice lab.",
    "ES": "Esta es una prueba de voz multilingüe generada desde Termux.",
}

for language, text in texts.items():
    model = TTS(language=language, device=device)
    speaker_ids = model.hps.data.spk2id
    speaker_key = list(speaker_ids.keys())[0]
    source_path = f"{output_dir}/tmp_{language}.wav"
    final_path = f"{output_dir}/openvoice_{language}.wav"
    model.tts_to_file(text, speaker_ids[speaker_key], source_path, speed=1.0)
    source_se = torch.load(f"checkpoints_v2/base_speakers/ses/{speaker_key}.pth",
        map_location=device)
    tone_color_converter.convert(
        audio_src_path=source_path,
        src_se=source_se,
        tgt_se=target_se,
        output_path=final_path,
        message="@OpenVoice",
    )
```

```
print(final_path)
```

Style-control tactic: In OpenVoice, voice identity and delivery style are partly separable. Keep the same reference clip for tone color, then vary the base TTS voice, prompt text, speed, punctuation, pauses, and language to shape emotion, cadence, and accent.

1. OpenVoice GitHub README: <https://github.com/myshell-ai/OpenVoice>

2. OpenVoice usage guide: <https://github.com/myshell-ai/OpenVoice/blob/main/docs/USAGE.md>

8. Clone with ElevenLabs IVC

The official ElevenLabs IVC quickstart uses `from elevenlabs.client import ElevenLabs`, loads `ELEVENLABS_API_KEY` from the environment, then calls `elevenlabs.voices.ivc.create(name=..., files=[BytesIO(...)])` and prints the returned voice ID.⁴

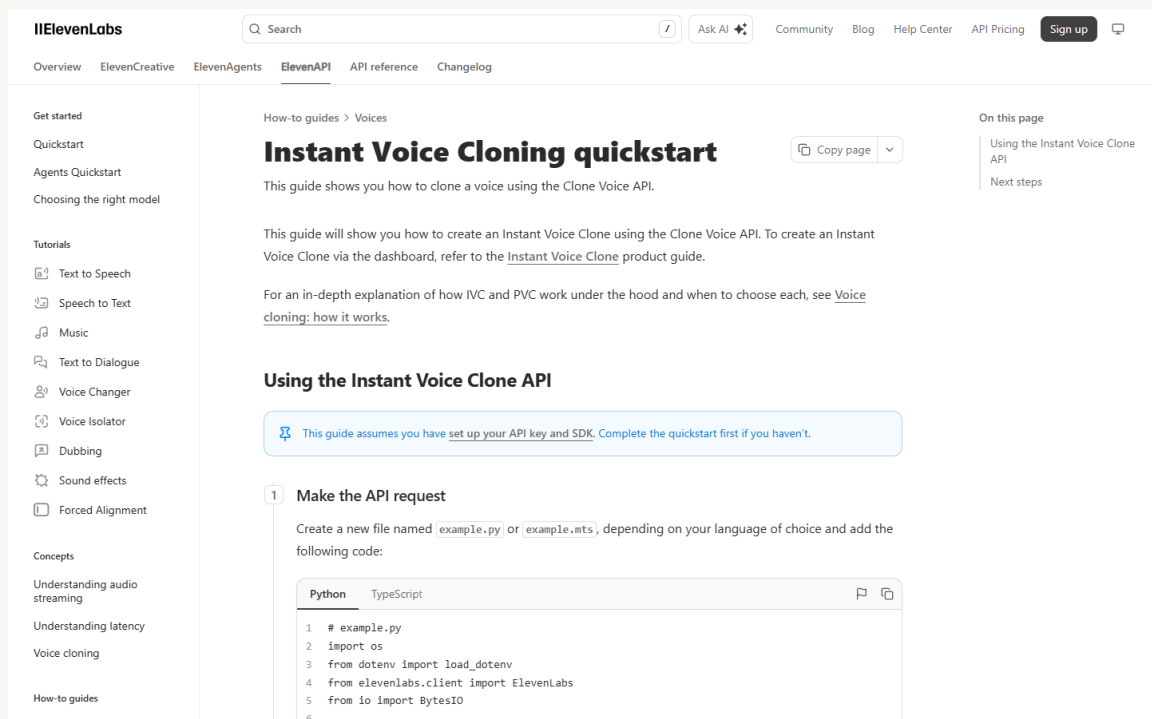


Figure 5. ElevenLabs Instant Voice Cloning quickstart showing the API flow.

```
cd ~/voice-lab
python -m venv .venv-eleven
source .venv-eleven/bin/activate
pip install --upgrade pip elevenlabs python-dotenv

# Store your API key locally. Do not commit this file to GitHub.
printf 'ELEVENLABS_API_KEY=replace_with_your_key\n' > .env
chmod 600 .env

# create_ivc.py
import os
from io import BytesIO
from pathlib import Path
from dotenv import load_dotenv
from elevenlabs.client import ElevenLabs

load_dotenv()
elevenlabs = ElevenLabs(api_key=os.getenv("ELEVENLABS_API_KEY"))

sample_paths = [
    Path("samples/ref_01_44k.wav"),
    Path("samples/ref_02_44k.wav"),
    Path("samples/ref_03_44k.wav"),
]

files = [
    BytesIO(path.read_bytes())
```



```

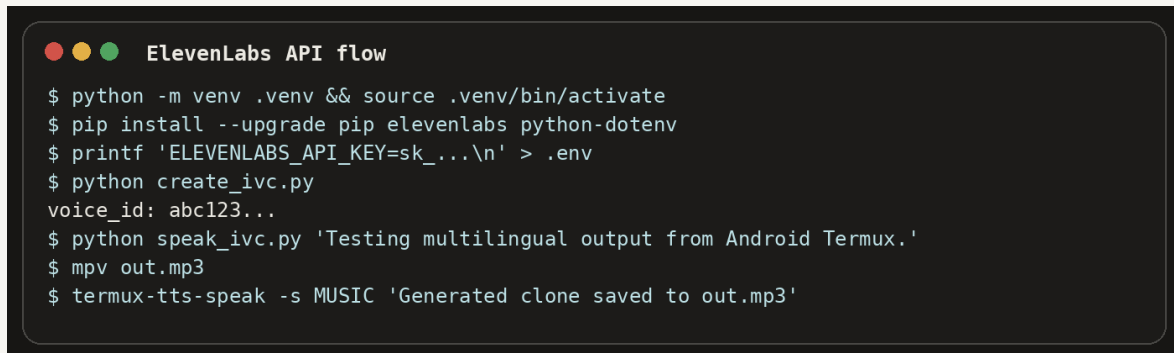
    for path in sample_paths
    if path.exists()
]

voice = elevenlabs.voices.ivc.create(
    name="Ron Termux Voice Lab",
    files=files,
)

print(voice.voice_id)

```

The ElevenLabs IVC endpoint accepts a multipart form with a required name and required list of audio recording files, and the docs include an optional `remove_background_noise` parameter that defaults to `false`.⁴



```

$ python -m venv .venv && source .venv/bin/activate
$ pip install --upgrade pip elevenlabs python-dotenv
$ printf 'ELEVENLABS_API_KEY=sk_...\n' > .env
$ python create_ivc.py
voice_id: abc123...
$ python speak_ivc.py 'Testing multilingual output from Android Termux.'
$ mpv out.mp3
$ termux-tts-speak -s MUSIC 'Generated clone saved to out.mp3'

```

Figure 6. Terminal-style screenshot of the ElevenLabs IVC workflow in Termux.

4. ElevenLabs IVC quickstart: <https://elevenlabs.io/docs/eleven-api/guides/how-to/voices/instant-voice-cloning>

9. Generate and play speech

ElevenLabs' text-to-speech quickstart installs `elevenlabs` and `python-dotenv`, creates an `ElevenLabs` client, and calls `text_to_speech.convert` with text, voice ID, model ID, and output format.⁶

```

# speak_ivc.py
import os
import sys
from dotenv import load_dotenv
from elevenlabs import VoiceSettings
from elevenlabs.client import ElevenLabs

load_dotenv()
elevenlabs = ElevenLabs(api_key=os.getenv("ELEVENLABS_API_KEY"))

VOICE_ID = os.getenv("ELEVENLABS_VOICE_ID", "paste_voice_id_here")
text = " ".join(sys.argv[1:]) or "The first move is what sets everything in motion."

response = elevenlabs.text_to_speech.convert(
    voice_id=VOICE_ID,
    output_format="mp3_22050_32",
    text=text,
    model_id="eleven_multilingual_v2",
    voice_settings=VoiceSettings(
        stability=0.35,
        similarity_boost=0.85,
        style=0.25,
        use_speaker_boost=True,
        speed=1.0,
    ),
)

with open("out/elevenlabs_clone.mp3", "wb") as f:
    for chunk in response:
        if chunk:
            f.write(chunk)

print("out/elevenlabs_clone.mp3")

# Play the generated clone
pkg install mpv

```

```
mpv out/elevenlabs_clone.mp3

# Use Android system TTS as a fast status cue, not as cloned voice playback.
termux-tts-speak -s MUSIC "Generated clone saved. Playing output now."
mpv out/elevenlabs_clone.mp3
```

The Termux TTS command speaks text supplied as arguments or standard input, and supports engine, language, region, variant, pitch, rate, and stream options such as `-s MUSIC`.¹¹

ElevenLabs’ save-to-file example writes each chunk returned by `text_to_speech.convert` into an MP3 file and uses `mp3_22050_32` for MP3 at 22.05 kHz and 32 kbps.⁷

6. ElevenLabs text-to-speech quickstart: <https://elevenlabs.io/docs/eleven-api/quickstart>

7. ElevenLabs streaming TTS guide: <https://elevenlabs.io/docs/eleven-api/guides/how-to/text-to-speech/streaming>

11. Termux TTS speak: <https://wiki.termux.com/wiki/Termux-tts-speak>

10. Test multilingual output

OpenVoice V2 natively supports English, Spanish, French, Chinese, Japanese, and Korean, while its quick-use list includes British/American/Indian/Australian English plus Spanish, French, Chinese, Japanese, and Korean.²

```
# multilingual_test.py for ElevenLabs cloned voice
from subprocess import run

tests = {
    "en": "London market update: risk is controlled, execution is patient.",
    "es": "Actualización del mercado: el riesgo está controlado.",
    "fr": "Mise à jour du marché: le risque reste maîtrisé.",
    "de": "Dies ist ein mehrsprachiger Test der geklonten Stimme.",
}

for lang, text in tests.items():
    run(["python", "speak_ivc.py", text], check=True)
    run(["cp", "out/elevenlabs_clone.mp3", f"out/test_{lang}.mp3"], check=True)
```

Test	Pass condition	Fix if it fails
Identity	Sounds recognizably like the reference speaker	Use cleaner/larger samples; avoid noisy phone recordings
Cadence	Pauses and rhythm feel natural	Rewrite punctuation; lower speed; use shorter clauses
Language	Pronunciation is intelligible in each language	Use matching model/language settings; test shorter text
Latency	Generation time fits workflow	Use ElevenLabs Flash models for interactive use; use local batch generation for OpenVoice
Playback	MP3/WAV plays reliably in Termux	Install mpv/ffmpeg; verify file path and Android volume stream

2. OpenVoice usage guide: <https://github.com/myshell-ai/OpenVoice/blob/main/docs/USAGE.md>

6. ElevenLabs text-to-speech quickstart: <https://elevenlabs.io/docs/eleven-api/quickstart>

7. ElevenLabs streaming TTS guide: <https://elevenlabs.io/docs/eleven-api/guides/how-to/text-to-speech/streaming>

11. Fine-tuning decisions

Use IVC when you need quick cloning from short samples, prototyping speed, or an assistant/trading-alert voice that can tolerate moderate variation. Use PVC-style fine-tuning when the voice is unique, the accent is not captured well by IVC, or the voice becomes a production brand asset that needs higher consistency.⁵

ElevenLabs states that IVC does not train a custom AI model, while PVC trains a dedicated model and is more accurate/customizable; the same overview says PVC fine-tuning generally takes hours and depends on queue and other factors.⁵

```
# Decision log template
Voice asset: Ron Termux Voice Lab
Purpose: trading notes / voice interface / multilingual tests
Consent: self-owned voice
IVC samples: 3 clips, 25 seconds each, clean room
IVC result: pass / retry / upgrade to PVC
OpenVoice result: pass / dependency retry needed / needs cleaner reference
Production label: AI-generated voice
```

5. ElevenLabs voice cloning overview: <https://elevenlabs.io/docs/eleven-creative/voices/voice-cloning>

12. Troubleshooting

Symptom	Likely cause	Fix
Termux:API command not found	Package missing or plugin not installed	Install Termux:API app from the same source as Termux, then run <code>pkg install termux-api</code> .
Microphone records silence	Android microphone permission not granted to Termux:API	Open Android App Info for Termux:API and allow Microphone. Re-run <code>termux-microphone-record</code> .
App/plugin install fails	Mixed APK signatures	Uninstall Termux and plugins, then reinstall all from one source only.
pkg install conflicts	Outdated package metadata	Run <code>pkg update && pkg upgrade</code> . Avoid Debian/Ubuntu repositories in native Termux.
pip build fails	Missing compiler or native library headers	Install <code>clang make cmake pkg-config rust libsndfile sox ffmpeg</code> ; upgrade <code>pip/wheel/setuptools</code> ; retry in a clean native venv.
OpenVoice import errors	Python/PyTorch version mismatch	Use a clean native venv; pin compatible versions where needed; capture the exact import error before changing multiple packages.
OpenVoice output is noisy/silent	Bad checkpoint layout, bad reference audio, or dependency mismatch	Re-extract checkpoints_v2, verify mono WAV, test official notebooks before custom scripts.
ElevenLabs 401/403	Missing/invalid API key or account permission	Check <code>.env</code> , export <code>ELEVENLABS_API_KEY</code> , verify account access.
Generated MP3 will not play	No player installed or wrong stream volume	Install <code>mpv</code> ; test <code>ffplay</code> ; raise MUSIC/media volume; use <code>termux-tts-speak -s MUSIC</code> for status cues.
GitHub accidentally receives keys	<code>.env</code> committed	Add <code>.env</code> to <code>.gitignore</code> immediately and rotate the API key.

8. Termux app installation notes: <https://github.com/termux/termux-app>

9. Termux package management: https://wiki.termux.com/wiki/Package_Management

10. Termux microphone record: <https://wiki.termux.com/wiki/Termux-microphone-record>

11. Termux TTS speak: <https://wiki.termux.com/wiki/Termux-tts-speak>

13. Build checklist

- [] Termux and Termux:API installed from the same source.
- [] Android microphone permission granted to Termux:API.
- [] Core packages installed: python, git, ffmpeg, termux-api.
- [] Voice-lab folders created under ~/voice-lab.
- [] Three or more clean voice clips recorded and converted with FFmpeg.
- [] OpenVoice repo cloned and installed in a native Termux Python venv.
- [] OpenVoice checkpoints extracted into checkpoints or checkpoints_v2 as appropriate.
- [] ElevenLabs SDK installed in a separate Python venv.
- [] ELEVENLABS_API_KEY stored in .env and protected with chmod 600.
- [] IVC voice created and voice_id saved.
- [] TTS generation script saves MP3 output into out/.
- [] Generated audio tested in English and at least one non-English language.
- [] termux-tts-speak used for Android status prompts, not confused with clone playback.
- [] Consent and labeling rules documented for every cloned voice.

Recommended final smoke test:

```
cd ~/voice-lab
source .venv-eleven/bin/activate
export ELEVENLABS_VOICE_ID="your_voice_id"
python speak_ivc.py "This is a final Android Termux voice-cloning smoke test."
termux-tts-speak -s MUSIC "Smoke test complete."
mpv out/elevenlabs_clone.mp3
```

- 4. ElevenLabs IVC quickstart: <https://elevenlabs.io/docs/eleven-api/guides/how-to/voices/instant-voice-cloning>
- 6. ElevenLabs text-to-speech quickstart: <https://elevenlabs.io/docs/eleven-api/quickstart>
- 7. ElevenLabs streaming TTS guide: <https://elevenlabs.io/docs/eleven-api/guides/how-to/text-to-speech/streaming>
- 11. Termux TTS speak: <https://wiki.termux.com/wiki/Termux-tts-speak>